Mixed-Width Asynchronous FIFO

User Manual – VHDL IP core with independent clock domains and different read / write data widths

Core	`mixed_width_async_FIFO` (VHDL)
Author	Emir Uruç
Clocks	Two independent clocks: `clk_wr` and `clk_rd`
Data widths	Configurable; width ratio must be a power of two. Simulated: 8→32, 32→8 and 8→8
Depth	$2^{\text{narrow_addr}}$ locations of the narrower width (default 16)
Clock domain crossing	Gray-coded pointers through N-stage flip-flop synchronizers
Simulation	Vivado XSim, six testbenches (three of them self-checking)

Author: Emir Uruç

Contents

1. Overview . . . 3

1.1 Features . . . 3

1.2 Supported configurations . . . 3

2. Interface . . . 3

2.1 Generics . . . 3

2.2 Ports . . . 3

3. Functional description . . . 4

3.1 Write and read operation . . . 4

3.2 Word order and data packing . . . 5

3.3 Full and empty . . . 5

3.4 Reset . . . 5

4. Architecture . . . 5

4.1 Block diagram . . . 5

4.2 Pointers . . . 6

4.3 Clock domain crossing and scaling . . . 6

4.4 Flag computation . . . 7

4.5 Memory access . . . 7

5. Integration . . . 8

5.1 Instantiation . . . 8

5.2 Usage rules . . . 8

5.3 Timing constraints . . . 8

6. Limitations and constraints . . . 8

7. Verification . . . 9

7.1 Results . . . 9

7.2 Not covered . . . 9

Appendix A. Internal signals . . . 10

1. Overview

The mixed-width asynchronous FIFO moves data between two unrelated clock domains and converts the data width at the same time. Typical uses are packing a byte stream into 32-bit words (8→32) or unpacking 32-bit words into a byte stream (32→8). With equal widths it behaves like a conventional asynchronous FIFO.

1.1 Features

- Independent write clock `clk_wr` and read clock `clk_rd`; no frequency or phase relationship is required.
- Different write and read widths. The wide port moves R = wide width / narrow width storage locations per access (R = 1, 2, 4, 8, ...).
- Parametrizable depth: $2^{\text{narrow_addr}}$ locations of the narrow width.
- Gray-coded pointers cross the clock boundary through N-stage flip-flop synchronizers. `full` is generated in the write domain and `empty` in the read domain.
- Plain VHDL. The only tool-specific item is the `ASYNC_REG` attribute (Xilinx) on the synchronizer registers.

1.2 Supported configurations

Write width	Read width	R	Mode	Status
8	32	4	write narrow, read wide	Simulated
32	8	4	write wide, read narrow	Simulated
8	8	1	equal widths	Simulated
other power-of-two ratios (e.g. 8→16, 8→64, 16→8)		2, 8, ...	same logic, other R	Supported by the design, not simulated

2. Interface

2.1 Generics

Generic	Default	Description
`wr_width`	8	Write data width in bits.
`rd_width`	32	Read data width in bits.
`narrow_addr`	4	Address bits of the narrow-width storage. Depth = $2^{\text{narrow_addr}}$ narrow words (4 → 16 locations).
`N`	3	Flip-flop stages in each synchronizer. At least 2 are needed for metastability protection; only N = 3 was simulated.

2.2 Ports

Port	Dir	Width	Description
clk_wr	in	1	Write clock. All write-side logic is rising-edge triggered.
rst_wr	in	1	Synchronous, active-high reset of the write side.
en_wr	in	1	Write request. A write is performed at a rising clk_wr edge when en_wr = '1' and full = '0'.
din	in	wr_width	Write data, sampled at the accepted write edge.
full	out	1	No write can be accepted (see section 3.3).
clk_rd	in	1	Read clock. All read-side logic is rising-edge triggered.
rst_rd	in	1	Synchronous, active-high reset of the read side.
en_rd	in	1	Read request. A read is performed at a rising clk_rd edge when en_rd = '1' and empty = '0'.
dout	out	rd_width	Read data. Registered: it is updated at the accepted read edge and keeps its value otherwise.
empty	out	1	No complete read word is available (see section 3.3).

3. Functional description

3.1 Write and read operation

- **Write:** data is stored at a rising edge of clk_wr in which en_wr = '1' and full = '0'. If full = '1' the request is ignored and nothing is overwritten.
- **Read:** data is taken at a rising edge of clk_rd in which en_rd = '1' and empty = '0'. If empty = '1' the request is ignored.
- **Registered output:** dout is a register. The data of an accepted read is visible after that clock edge (one clock of read latency). This is a standard FIFO, not a first-word-fall-through FIFO.
- The flags depend only on registered state, not on en_wr or en_rd.

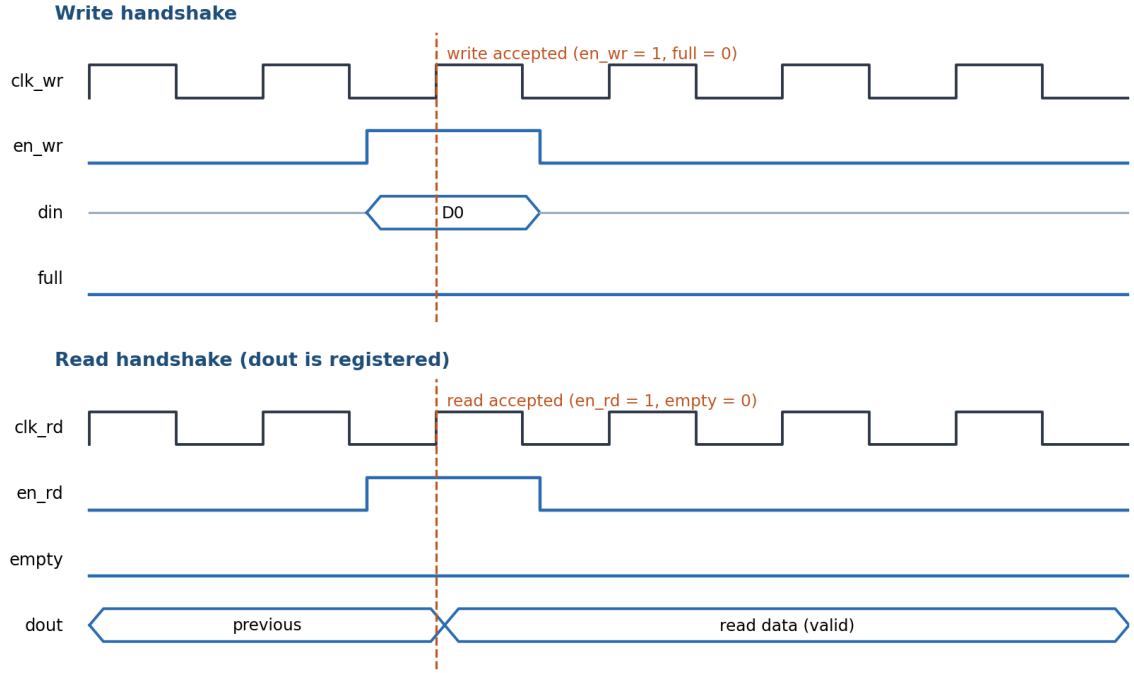


Figure 1 – Write and read handshake. Enables are driven after the falling edge and sampled at the rising edge.

3.2 Word order and data packing

Narrow word *i* of a wide word occupies bits [(*i*+1)·narrow_width − 1 : *i*·narrow_width]. Word 0 is written first and read first, so the first narrow word is always the **least significant** part of the wide word, on both the write and the read side.

Example	Writes	Reads
8→32	0x01, 0x02, 0x03, 0x04 (four narrow writes)	one read returns `0x04030201`
32→8	one write of `0xABCD1234`	four reads return 0x34, 0x12, 0xCD, 0xAB

3.3 Full and empty

Configuration	full (write domain)	empty (read domain)
8→32 (write narrow, read wide)	All $2^{\text{narrow_addr}}$ locations hold unread data.	Fewer than R narrow words are stored. A partly filled wide word cannot be read.
32→8 (write wide, read narrow)	Fewer than R locations are free. A wide write needs R free locations.	The read pointer has caught up with the write pointer.
8→8 (equal widths)	All locations used.	No location used.

- **Latency:** a pointer needs about N cycles of the receiving clock to cross. After a write, `empty` is released about N `clk_rd` cycles later; after a read, `full` is released about N `clk_wr` cycles later.
- **Conservative flags:** the synchronized pointer is always slightly old, so the flags never allow an overflow or underflow. They may stay asserted a few cycles longer than strictly necessary.
- **Wide writes:** in 32→8 with the default depth (16 bytes) the FIFO holds four wide words. After it is full, `full` is released only after four narrow reads plus the synchronizer delay.

3.4 Reset

- Resets are synchronous and active high. `rst_wr` clears the write pointer and the synchronizer that feeds the write side; `rst_rd` clears the read pointer, the synchronizer that feeds the read side and `dout`.
- The memory contents are not cleared.
- After reset: `empty` = '1' and `full` = '0'.
- The two resets are independent. Assert both together and hold them for several cycles of both clocks before the first access; if only one side is reset the pointers become inconsistent.

4. Architecture

4.1 Block diagram

The core consists of the blocks listed below. Figure 2 shows how they are connected and which signals cross the clock boundary.

Block	Function	Why it is needed
Write / read pointer	Binary counter with one extra wrap bit, plus its Gray-coded copy.	Counts accepted accesses. The Gray copy is the value that crosses the clock boundary.
Memory	2^{narrow_addr} locations, each narrow_width bits wide.	The wide port reads or writes R consecutive locations in a single clock.
Synchronizers	Chain of N flip-flops per direction (`ASYNC_REG`).	Reduces the probability of a metastable value reaching the logic.
Gray → binary, scaling	Receiving domain converts the pointer back to binary and, if it comes from the wide side, multiplies it by R.	Both pointers must be in the same unit (narrow words) before they can be compared.
Occupancy / flag logic	Computes stored and free words and generates `full` / `empty`.	Applies the R-word thresholds of the wide port.

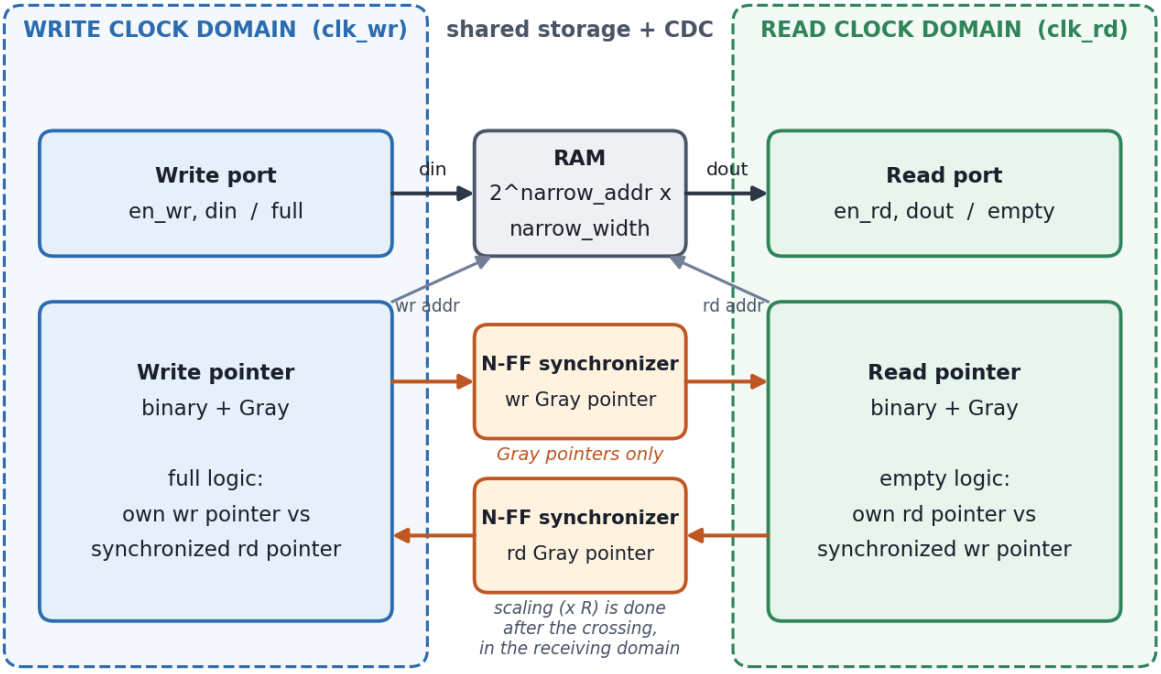


Figure 2 – Block diagram. Only Gray-coded pointers cross between the clock domains.

4.2 Pointers

Each side has a binary pointer with one extra most significant bit. The pointers therefore wrap after twice the depth: equal pointers mean empty, pointers that differ by exactly the depth mean full. The wide-side pointer counts *wide words*, so it needs $\log_2(R)$ address bits less than the narrow-side pointer.

Configuration	Write pointer	Read pointer	Counts in units of
8→32 (narrow_addr = 4, R = 4)	5 bits	3 bits	write: narrow words, read: wide words
32→8 (narrow_addr = 4, R = 4)	3 bits	5 bits	write: wide words, read: narrow words
8→8 (narrow_addr = 4)	5 bits	5 bits	narrow words

4.3 Clock domain crossing and scaling

A Gray code is safe to synchronize because only one bit changes per step: whichever way the synchronizer samples that bit, it sees either the old or the new pointer value. This property holds only for a counter that steps by 1.

A pointer of the wide side is compared in narrow-word units, i.e. multiplied by R. If this multiplication were done *before* the Gray conversion, the pointer would step by R and its Gray code would change two bits per step (table below, R = 4). A synchronizer could then capture one new and one old bit and produce a value the pointer never had. For this reason the Gray code is always generated from the unscaled counter and the scaling is done in the receiving domain, after the synchronizer and the Gray-to-binary conversion.

Wide pointer k	Narrow units (k · 4)	Gray of scaled value	Bits changed	Gray of k (used)	Bits changed
0	0	00000	–	000	–
1	4	00110	2	001	1
2	8	01100	2	011	1
3	12	01010	2	010	1
4	16	11000	2	110	1

Data path: wide counter (+1) → Gray → N-stage synchronizer → Gray-to-binary → append $\log_2$(R) zeros (× R) → comparison.

4.4 Flag computation

Configuration	Flag	Domain	Computation
8→32	empty	read	stored = synchronized write pointer − own read pointer × R; empty when stored < R
8→32	full	write	stored = own write pointer − synchronized read pointer × R; full when stored = depth
32→8	full	write	stored = own write pointer × R − synchronized read pointer; free = depth − stored; full when free < R
32→8	empty	read	empty when own read pointer = synchronized write pointer × R
8→8	full / empty	write / read	Gray comparison: full when the write Gray pointer equals the synchronized read Gray pointer with its two top bits inverted; empty when the read Gray pointer equals the synchronized write Gray pointer

All subtractions use the extra wrap bit, so they stay correct when the pointers wrap around (modulo 2^{narrow_addr+1}).

4.5 Memory access

In wide-write mode one accepted write stores R consecutive locations starting at the write pointer × R; in wide-read mode one accepted read collects R consecutive locations into `dout`. Addresses wrap inside the 2^{narrow_addr} locations. Because several locations are accessed in one clock with a computed address, the memory is likely mapped to distributed RAM or registers instead of block RAM; check the utilization report for your target.

5. Integration

5.1 Instantiation

```
u_fifo : entity work.mixed_width_async_FIFO
    generic map (
        wr_width    => 8,
        rd_width    => 32,
        narrow_addr => 4,
        N           => 3
    )
    port map (
        clk_wr => clk_wr,  rst_wr => rst_wr,  en_wr => en_wr,
        din    => din,     full   => full,
        clk_rd => clk_rd,  rst_rd => rst_rd,  en_rd => en_rd,
        dout   => dout,    empty  => empty
    );
```

5.2 Usage rules

- Release both resets before the first access.
- A write has happened if `en_wr` = '1' and `full` = '0' at the rising edge. Check `full` before counting a word as written; a rejected word must be offered again.
- A read has happened if `en_rd` = '1' and `empty` = '0' at the rising edge. Take `dout` one clock later.
- Sending requests while a flag is asserted is harmless; they are ignored.

5.3 Timing constraints

The two clocks are asynchronous. Declare them as such and keep the Gray pointer paths short. Example for Vivado:

```
create_clock -period 10.000 [get_ports clk_wr]
create_clock -period 7.500  [get_ports clk_rd]
set_clock_groups -asynchronous -group [get_clocks clk_wr] -group [get_clocks clk_rd]
```

> **Note**
> With *set_clock_groups -asynchronous* the paths between the clocks are not timed at all. Alternatively bound the Gray pointer paths with *set_max_delay -datapath_only*. Check the vendor documentation before combining both styles.

6. Limitations and constraints

Item	Description
Width ratio	The ratio must be a power of two and the wide width a multiple of the narrow width (R = 1, 2, 4, 8, ...). This is not checked in the code; other ratios give wrong results.
Tested parameters	Only narrow_addr = 4 and N = 3 were simulated. narrow_addr should be larger than $\log_2(R)$; very small values and N < 2 were not tested.
Depth	Fixed to 2^{narrow_addr} narrow words (e.g. 16 bytes = four 32-bit words).
Partial wide words	A partly filled wide word is never flushed to the read side. There are no almost-full / almost-empty flags.
Resets	The two resets are independent; assert them together and hold them long enough (section 3.4).
Memory mapping	Multi-location access per clock makes block RAM inference unlikely (section 4.5).
VHDL version	The constants `wr_ptr_addr` and `rd_ptr_addr` use a conditional expression (VHDL-2019), which Vivado accepted. Replace it by a function for tools without support.
CDC constraints	Clock-crossing constraints must be added by the user (section 5.3). `ASYNC_REG` is a Xilinx attribute.
Verification level	Verified in simulation only: no formal CDC analysis and no hardware test.

7. Verification

The design was simulated in Vivado XSim with a 100 MHz write clock (10 ns) and a 133 MHz read clock (7.5 ns). The three streaming testbenches check every received byte and end with a PASS / FAIL message and a watchdog timeout.

Testbench	Direction	Scenario	Check
`tb_mixed_FIFO_1`	8→32	Four narrow writes (0..3), one wide read	Waveform: dout = 0x03020100
`tb_mixed_FIFO_2`	32→8	One write of 0xABCD1234, four narrow reads	Waveform: 0x34, 0x12, 0xCD, 0xAB; empty after the 4th read
`tb_mixed_FIFO_3`	32→8	FIFO filled with wide words, a further write while full, 16 reads	Waveform: reads 0x01..0x10, rejected write never appears
`tb_mixed_FIFO_4`	32→8	25 wide writes and 100 reads running concurrently; full cycles repeatedly; pointers wrap	Self-checking (PASS)
`tb_mixed_FIFO_5`	8→32	100 narrow writes and 25 wide reads, slow reader; full cycles repeatedly; pointers wrap	Self-checking (PASS)
`tb_mixed_FIFO_6`	8→8	100 bytes with a slow reader; full cycles repeatedly; pointers wrap	Self-checking (PASS)

7.1 Results

- All six testbenches ran to completion; the self-checking ones ended with PASS (100 bytes received in the correct order).
- The checker was shown to detect errors: a deliberately wrong expected value stopped the simulation with FAIL at the first byte.
- Waveform inspection: the Gray pointers change one bit per step, including at the wrap-around; the scaled pointers wrap correctly; the stored-word counts never exceed the depth.

7.2 Not covered

- Random enable timing and random clock ratios (only fixed 10 ns / 7.5 ns clocks were used).
- Other width ratios (R = 2, 8, ...), other `narrow_addr` and N values.

- The empty boundary inside a long mixed-width stream: in the streaming tests the reader was never starved after the start.
- Reset during operation, formal CDC analysis and hardware tests.

Appendix A. Internal signals

Signal	Meaning
`wr_ptr_bin / rd_ptr_bin`	Binary pointers of the own clock domain (with wrap bit).
`wr_ptr_bin_next / rd_ptr_bin_next`	Pointer value after the current access.
`wr_ptr_gray / rd_ptr_gray`	Gray-coded pointers; the only values that cross the clock boundary.
`wr_ptr_gray_sync / rd_ptr_gray_sync`	Output of the synchronizers (pointer of the other domain).
`wr_gray_to_bin / rd_gray_to_bin`	Synchronized pointers converted back to binary.
`wr_ptr_bin_fe_scaled / rd_ptr_bin_fe_scaled`	Own pointers in narrow-word units (also used as RAM address).
`wr_gray_to_bin_scaled / rd_gray_to_bin_scaled`	Synchronized wide-side pointers in narrow-word units.
`difference`	Stored narrow words (write pointer − read pointer), computed in one domain.
`r_space`	Free narrow words (depth − difference).
`wr_fill`	Stored narrow words as seen by the write side (8→32 only).
`full_o / empty_i`	Internal flags that drive the ports `full` and `empty`.